

# Build a Hyperlane ETH Warp Route (Base → your chain)

A builder-facing guide to a **self-hosted** Hyperlane warp that locks **native ETH on Base** and mints **synthetic ETH 1:1 on your chain**. The worked example uses Base ETH → Lightchain, but any Hyperlane-registered chain works. This is the sibling of the [USDC warp guide](#) — the only real difference is the origin side is a chain's **native coin** ( `type: native` ), not an ERC-20.

**Read this first.** This is a route *you operate* — your own validator signs, your own relayer delivers. It is **not** an official or public bridge, and **not** a one-click UI. The default security is **threshold-1** (a single validator whose key can authorize mints), so it is a single point of trust. Keep the agents online, protect the key, and add validators before calling it trustless. Liquidity is only whatever you bridge — this is working plumbing, not a market on-ramp.

**Why synthetic ETH and not WETH?** WETH wraps a chain's *own native* coin. On a chain whose native coin is **not** ETH (e.g. Lightchain, native = LCAI), a "WETH" contract has nothing local to wrap and stays empty. Synthetic ETH is **bridged** — it carries real ETH value locked on Base. For onboarding holders from outside your chain, synthetic ETH is the working token; WETH is not.

## 0. Prerequisites

- **Funds:** ops wallet with **ETH on Base** (deploy + gas + the bridged amount) and **native gas on your destination chain** (deploy + relay).
- **Host:** an always-on machine with **Docker** (validator + relayer run 24/7).
- **Tooling:** Node.js + the Hyperlane CLI ( `@hyperlane-xyz/cli` , pin a known version) and the agent image `ghcr.io/hyperlane-xyz/hyperlane-agent` .
- **Key:** a deployer/signer keyfile, permissions `0600` , **never committed to git**. Load it into the CLI via the `HYP_KEY` env var — never echo or paste the value.

## 1. Register your destination chain with Hyperlane

Add your chain to the Hyperlane registry: a name, its **chainId / domain**, an **RPC**, and a deployed **Mailbox** (+ ISM factory). If your chain has no Hyperlane core yet, run `hyperlane core deploy` first. Agent chain metadata lives in `configs/agent-config.json` .

*Example (Lightchain):* name `lightchainai` · domain `9200` · RPC `https://rpc.mainnet.lightchain.ai` · Mailbox `0x142a9CEf00ACcAddB76283c49A1Bf37f20c1F00e` .

## 2. Write the warp recipe ( `warp-route-deployment.yaml` )

Base side = **native** (wraps native ETH — no `token` field). Your side = **synthetic** ETH that mints when a Base lock is delivered. Set `owner` to your own ops wallet (not a hardware/treasury wallet).

```
# Base native ETH → <yourchain> synthetic ETH. Owner = your anon deployer.
base:
  type: native                                # native ETH – NO token field
  owner: "0xYOUR_OPS_WALLET"
  mailbox: "0xeA87ae93Fa0019a82A727bfd3eBd1cFCa8f64f1D" # Base mailbox
  decimals: 18
  name: "Ether"
  symbol: "ETH"
yourchain:                                    # e.g. lightchainai
  type: synthetic
  owner: "0xYOUR_OPS_WALLET"
  mailbox: "0xYOUR_CHAIN_MAILBOX"
  decimals: 18
  name: "Synthetic Ether"
  symbol: "ETH"
```

**Native vs ERC-20:** because ETH is Base's native coin, the origin side is `type: native` with **no token address** — that is the one change from the USDC recipe. Decimals are **18** on both sides. One warp route = one token, so this is a separate route from any USDC or LCAI warp — **rails never mix**.

### 3. Deploy the route

```
export PATH="$HOME/.npm-global/bin:$PATH"
# Load HYP_KEY from your local keyfile – do NOT print it
cd ~/.config/<your-deploy-dir>/warp-workdir
hyperlane warp deploy -y -w ETH/base-yourchain --verbosity info
unset HYP_KEY
```

**Output (yours will differ):** a `HypNative` collateral router on Base (locks ETH) and a synthetic `HypERC20` on your chain (mints ETH). The CLI enrolls the two routers to each other and writes artifacts to `~/.hyperlane/`.

**Deploy gotcha (real, seen twice).** Public Base RPCs ( `mainnet.base.org` , `publicnode` , `llamarpc` ) may **403 deploy-class calls** from an ops host — the deploy half-lands (synthetic proxies created on your chain, Base side fails). Fix: deploy through <https://base.gateway.tenderly.co> , then, if your chain side already deployed, **enroll against the existing synthetic proxy** rather than running a third full deploy.

### 4. Choose your ISM (security module)

Point the synthetic token at an ISM that decides which messages are valid. The simplest bootstrap is a **TrustedRelayerIsM** (you relay). For self-verifying delivery, use a **MessageIdMultisigIsM** with your validator announced on Base:

```
hyperlane ism deploy --chain yourchain -i ism-config.yaml -y --verbosity info
```

A live example runs **MessageIdMultisigIsm, threshold 1**, validator set = a single Base-announced validator. **Threshold 1 is a single trust point** — add validators to harden it.

## 5. Run validator + relayer (Docker, always-on)

```
cd ~/.config/<your-deploy-dir>
CONFIG_FILES="$(pwd)/configs/agent-config.json"
SIGDIR="$(pwd)/tmp/validator-signatures"
# KEY = load your signer into a shell var from the keyfile; never echo it

docker run -d --name hyp-validator-base --restart unless-stopped \
  --mount type=bind,source="$CONFIG_FILES",target=/config/agent-config.json,readonly \
  --mount type=bind,source="$(pwd)/db_validator",target=/hyperlane_db \
  --mount type=bind,source="$SIGDIR",target=/tmp/validator-signatures \
  -e CONFIG_FILES=/config/agent-config.json \
  ghcr.io/hyperlane-xyz/hyperlane-agent ./validator \
  --db /hyperlane_db --originChainName base \
  --checkpointSyncer.type localStorage \
  --checkpointSyncer.path /tmp/validator-signatures \
  --validator.key "$KEY"

docker run -d --name hyp-relayer --restart unless-stopped \
  --mount type=bind,source="$CONFIG_FILES",target=/config/agent-config.json,readonly \
  --mount type=bind,source="$(pwd)/db_relayer",target=/hyperlane_db \
  --mount type=bind,source="$SIGDIR",target=/tmp/validator-signatures,readonly \
  -e CONFIG_FILES=/config/agent-config.json \
  ghcr.io/hyperlane-xyz/hyperlane-agent ./relayer \
  --db /hyperlane_db --relayChains base,yourchain \
  --allowLocalCheckpointSyncers true --defaultSigner.key "$KEY"
```

**Key hygiene:** passing `--validator.key` on the command line leaves the hex visible via `docker inspect`. Prefer env/file injection with tight permissions. Keep the relayer wallet funded on your chain for gas. **Delivery depends on these agents being online** — if the host is off, mints wait until it comes back.

## 6. Test send — a few dollars end-to-end before you trust it

You can send from the CLI, or (as in the live route) call `transferRemote` directly on the Base router. The router is a **HypNative** collateral contract, so the call is payable and `msg.value` must cover **the bridged amount + interchain gas**:

```
transferRemote(destinationDomain, bytes32(recipient), amount)
msg.value = amount + quoteGasPayment(destinationDomain)
```

One transaction on Base locks the ETH; synthetic ETH mints to the **same recipient address** on your chain once the relayer delivers — usually a couple of minutes. A healthy send shows as processed on [explorer.hyperlane.xyz](https://explorer.hyperlane.xyz). Anyone bridging afterward needs Base ETH + Base gas **and** your delivery online.

## 7. Wire it into a public app (optional)

To give users a one-click path instead of a raw contract call, add a route in your frontend that builds the same `transferRemote` call. In the live example the send is a small inline function — it computes `msg.value = amount + quoteGasPayment(9200)` and calls the Base router — kept **separate** from the app's other bridge rails so ETH never travels down a USDC or LCAI warp. The synthetic ETH that lands can then be accepted as a payment token across your apps alongside your other tokens.

## Reference addresses

| Role                               | Address                                                 | Reuse?                                 |
|------------------------------------|---------------------------------------------------------|----------------------------------------|
| Mailbox (Base)                     | <code>0xeA87ae93Fa0019a82A727bfd3eBd1cFCa8f64f1D</code> | Universal — use as-is                  |
| ValidatorAnnounce (Base)           | <code>0x182E8d7c5F1B06201b102123FC7dF0EaeB445a7B</code> | Universal — use as-is                  |
| Mailbox (Lightchain example)       | <code>0x142a9CEf00ACcAddB76283c49A1Bf37f20c1F00e</code> | Swap for your chain's mailbox          |
| Base native collateral (HypNative) | <i>generated by your deploy</i>                         | Yours will differ                      |
| Synthetic ETH (HypERC20)           | <i>generated by your deploy</i>                         | Yours will differ                      |
| ISM / validator                    | <i>your own values</i>                                  | Yours will differ                      |
| Owner / signer                     | <i>your ops wallet</i>                                  | Yours — never a hardware/ treasury key |

Native ETH has no token contract on Base (it's the gas coin), so there is nothing to list as a "token" address the way USDC has one — that's why the origin side is `type: native`.

## Security checklist

---

1. Threshold-1 ISM means the validator key can authorize mints — protect it and add validators before calling it trustless.
  2. Inject keys via env/files with tight permissions; avoid leaving hex in the Docker `Cmd` (visible via `docker inspect`).
  3. Never commit the deployer keyfile, agent keys, or API keys. Keep the deploy directory out of git.
  4. Hyperlane CLI flags shift between versions — treat command *shapes* as guidance and confirm against current CLI docs before running.
  5. Keep rails isolated: one warp route per token. Don't route ETH through a USDC or LCAI warp.
  6. Don't market liquidity: synthetic ETH on a new chain is working plumbing, not a liquid on-ramp.
- 

*Reference: Hyperlane docs (warp routes, ISMs, agents) at [docs.hyperlane.xyz](https://docs.hyperlane.xyz). The Hyperlane mailbox and validator-announce addresses are public infrastructure; every wallet, ISM, collateral, and synthetic address in your own route is generated at deploy time and will differ from any example here.*